

Wyjaśnialne wykrywanie włamań sieciowych: analiza SHAP dla modeli Random Forest i XGBoost na zbiorach UNSW-NB15 oraz CIC-IoT2023

Dawid Budzyński, Filip Budzyński

Zespół 13, Systemy i Protokoły Zabezpieczeń w Cyberprzestrzeni (SPZC)
Politechnika Warszawska, Wydział Elektroniki i Technik Informacyjnych

Streszczenie—Modele uczenia maszynowego osiągają wysoką skuteczność w wykrywaniu włamań sieciowych (NIDS), ale ich nieprzejrzystość ogranicza zaufanie analityków bezpieczeństwa. W pracy wykorzystano wyjaśnialną sztuczną inteligencję (XAI), inspirowaną pracą Arreche i in. (2024), i zastosowano ją do wieloklasowej klasyfikacji ruchu sieciowego za pomocą modeli Random Forest oraz XGBoost. Wyjaśnienia generowane są metodą SHAP (TreeSHAP), zarówno globalnie, jak i lokalnie. Przeprowadzono eksperymenty na dwóch zbiorach danych: UNSW-NB15 oraz CIC-IoT2023. Zbadano trzy zagadnienia: (i) skuteczność klasyfikacji obu modeli, (ii) zgodność rankingów najważniejszych cech wskazywanych przez SHAP pomiędzy Random Forest a XGBoost, oraz (iii) spójność wyjaśnień pomiędzy zbiorami danych. Wykazano, że oba modele osiągają porównywalną skuteczność (dokładność do 83,1% i F1 do 0,832 na zbiorze CIC-IoT2023). Oba modele wskazują na spójność tych samych cech (zgodność Jaccarda rankingów do 0,67), a kluczową rolę odgrywają atrybuty związane z wolumenem bajtów, rozmiarem pakietów oraz czasem trwania połączenia, zgodnie z obserwacjami z literatury. Do walidacji poprawności implementacji odtworzono ten sam potok na zbiorze NSL-KDD wykorzystanym w pracy bazowej, uzyskując dokładność 99,6%, na poziomie wyników oryginalnych, co potwierdza, że niższa skuteczność na zbiorach UNSW-NB15 i CIC-IoT2023 wynika z ich trudności, a nie z błędów implementacji. Ograniczenie zbioru cech do 15 najważniejszych wg. SHAP zachowuje skuteczność modeli przy znaczącej redukcji wymiarowości.

Index Terms—wyjaśnialna sztuczna inteligencja, XAI, SHAP, wykrywanie włamań, NIDS, Random Forest, XGBoost, UNSW-NB15, CIC-IoT2023, NSL-KDD

I. WPROWADZENIE

Sieciowe systemy wykrywania włamań (ang. *Network Intrusion Detection Systems*, NIDS) stanowią jeden z podstawowych mechanizmów obronnych współczesnych sieci. Rosnąca liczba i złożoność ataków sprawiły, że klasyczne metody sygnaturowe są coraz częściej zastępowane modelami uczenia maszynowego, które potrafią wykrywać również, wcześniej nie spotykane warianty. Modele te osiągają bardzo wysoką skuteczność, jednak ich działanie pozostaje dla analityka nieprzejrzyste: decyzja o zaklasyfikowaniu ruchu jako atak nie jest opatrzona zrozumiałym uzasadnieniem. Brak wyjaśnialności obniża zaufanie do systemu, utrudnia audyt oraz reakcję na incydenty [1].

Odpowiedzią na ten problem jest wyjaśnialna sztuczna inteligencja (ang. *Explainable AI*, XAI), której celem jest dostarczenie zrozumiałych uzasadnień predykcji modeli. W

kontekście NIDS XAI pozwala odpowiedzieć na pytanie, które cechy ruchu sieciowego zadecydowały o wykryciu ataku, a tym samym zweryfikować poprawność modelu i wesprzeć pracę analityka.

Niniejsza praca nawiązuje do metodyki XAI-IDS zaproponowanej przez Arreche i in. [1], w której osiem modeli typu czarna skrzynka poddano analizie metodami SHAP oraz LIME na trzech zbiorach danych. W niniejszym projekcie skupiono się na dwóch reprezentatywnych modelach drzewiastych (Random Forest [2] oraz XGBoost [3]) i wykorzystano metodę SHAP [4], [5] do wyjaśnienia ich decyzji. W odróżnieniu od pracy bazowej, która oceniała modele m.in. na przestarzałym zbiorze NSL-KDD, główne eksperymenty przeprowadzono na dwóch nowszych i zróżnicowanych zbiorach: UNSW-NB15 [6] (ruch ogólnosieciowy) oraz CIC-IoT2023 [7] (ruch Internetu Rzeczy). Dodatkowo, w celu walidacji poprawności implementacji, odtworzono ten sam potok na zbiorze NSL-KDD wykorzystanym w pracy bazowej i porównano uzyskane wyniki bezpośrednio z raportowanymi przez autorów.

Wkład pracy obejmuje:

- implementację powtarzalnego potoku wykrywania włamań z wyjaśnieniami SHAP dla modeli Random Forest i XGBoost w trybie wieloklasowym;
- empiryczne porównanie zgodności rankingów najważniejszych cech pomiędzy dwoma modelami na tym samym zbiorze;
- analizę spójności wyjaśnień pomiędzy dwoma różnymi zbiorami danych;
- porównanie uzyskanych wyników z wartościami raportowanymi w pracy bazowej oraz badanie wpływu selekcji 15 najważniejszych cech na skuteczność.

II. PRACA POWIĄZANA

Zastosowanie uczenia maszynowego w wykrywaniu włamań jest szeroko opisane w literaturze. Klasyczne zbiory KDD'99 oraz NSL-KDD [8] przez lata stanowiły standardowy punkt odniesienia, jednak ich wiek (ponad 25 lat) i nierealistyczny rozkład ruchu czynią je nieadekwatnymi do oceny współczesnych systemów. W odpowiedzi powstały nowsze zbiory, m.in. UNSW-NB15 [6], łączący realny ruch sieciowy z syntetycznymi atakami, oraz CIC-IoT2023 [7],

Tabela I
CHARAKTERYSTYKA ZBIORÓW DANYCH (PO WSTĘPNYM PRZETWARZANIU).

Zbiór	Cechy	Klasy	Próbki (tren./test)
UNSW-NB15	42	10	175 341 / 82 332
CIC-IoT2023	39	8	150 002 / 268 463
NSL-KDD [†]	40	5	103 961 / 44 556

[†] Zbiór walidacyjny (reprodukcja wyników pracy bazowej); nie wchodzi do głównej analizy. Liczba cech po usunięciu kolumn stałych.

obejmujący ataki na urządzenia IoT, w tym botnet Mirai i liczne warianty DDoS.

Tematyka wyjaśnialności modeli NIDS zyskuje na znaczeniu. SHAP [4], oparty na wartościach Shapleya z teorii gier, przypisuje każdej cesze wkład w konkretną predykcję i posiada efektywny wariant dla modeli drzewiastych: TreeSHAP [5]. Alternatywą jest LIME [9], dostarczający lokalnych przybliżeń liniowych. Praca bazowa Arreche i in. [1] systematyzuje zastosowanie tych technik w NIDS, a jej rozszerzenie [10] pokazuje, że cechy wybrane na podstawie SHAP pozwalają znacząco zredukować wymiarowość bez utraty skuteczności. Nasza praca powtarza i weryfikuje te obserwacje na innych, nowszych zbiorach danych.

III. METODYKA

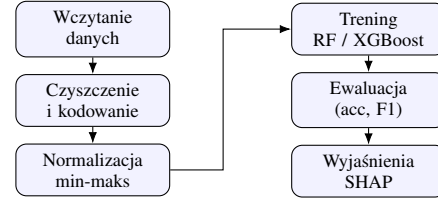
A. Zbiory danych

W głównej części eksperymentów wykorzystano dwa współczesne zbiory danych (UNSW-NB15 oraz CIC-IoT2023) zgodnie z zaleceniem rezygnacji z przestarzałych zbiorów rodziny KDD jako materiału głównego. Dodatkowo, wyłącznie w celu walidacji reprodukcyjnej (porównania z wynikami pracy bazowej), uruchomiono ten sam potok na zbiorze NSL-KDD. Wszystkie trzy zbiory opisano w Tabeli I.

UNSW-NB15 zawiera 42 cechy opisujące ruch sieciowy (m.in. czas trwania, liczbę i wolumen bajtów, statystyki TTL i liczniki połączeń) oraz etykietę `attack_cat` z dziesięcioma klasami (ruch normalny i dziewięć kategorii ataków). Skorzystano z oficjalnego podziału na zbiór treningowy i testowy.

CIC-IoT2023 opisuje ruch w sieci 105 urządzeń IoT. Po pogrupowaniu 33 typów ataków otrzymano osiem klas: *Benign*, *DDoS*, *DoS*, *Mirai*, *Recon*, *Spoofing*, *Web-based* oraz *BruteForce*. Zbiór opisuje 39 cech (m.in. statystyki rozmiaru pakietów, tempo transmisji, czas między pakietami IAT oraz flagi TCP).

NSL-KDD (zbiór walidacyjny) zawiera 41 cech ruchu, których etykiety odwzorowano na pięć kanonicznych kategorii: *Normal*, *DoS*, *Probe*, *R2L* oraz *U2R*. Aby wiernie odtworzyć protokół pracy bazowej, połączono oficjalne pliki KDDTrain+ i KDDTest+, a następnie podzielono je losowo (warstwowo) w proporcji 70/30, zamiast korzystać z trudnego oficjalnego podziału KDDTest+ (znanego z rozbieżności rozkładów). Zbiór ten nie wchodzi do głównej analizy i służy wyłącznie do porównania liczbowego z artykułem.



Rysunek 1. Potok eksperymentalny: od surowych danych do wyjaśnień SHAP.

B. Potok przetwarzania

Implementacja, wzorowana na metodyce z [1], realizuje następujące kroki (Rys. 1): wczytanie danych, usunięcie kolumn stałych i identyfikatorów, kodowanie cech kategorycznych (etykietowe), usunięcie wartości brakujących i nieskończonych, normalizacja min-maks do przedziału $[0, 1]$ oraz podział na zbiór treningowy i testowy. Kodowanie etykietowe zamienia wartości tekstowe cech kategorycznych (np. `protocol_type` czy `service`) na kolejne liczby całkowite, co jest wymagane przez wykorzystywane modele; dla modeli drzewiastych wprowadzony w ten sposób sztuczny porządek wartości ma przy tym znikomy wpływ na skuteczność. Niezrównoważenie klas skompensowano wagami klas (`class_weight/sample_weight`) zamiast nadpróbkowania, co upraszcza potok przy zachowaniu porównywalnego efektu. Ze względu na rozmiar zbioru CIC-IoT2023 zbiór treningowy próbkowano w sposób warstwowy do 150 000 rekordów.

C. Modele

Zastosowano dwa modele drzewiaste reprezentujące dwa odmienne podejścia: *bagging* oraz *boosting*. Random Forest [2] skonfigurowano ze 100 drzewami i ważeniem klas `balanced`. XGBoost [3] wytrenowano z 200 drzewami, maksymalną głębokością 8 i współczynnikiem uczenia 0,3, stosując wagi próbek równoważące klasy. Implementację oparto na bibliotekach `scikit-learn` [11] oraz `xgboost`.

D. Wyjaśnienia SHAP

Do wyjaśnień wykorzystano TreeSHAP [5], dokładny i wydajny algorytm obliczania wartości SHAP dla modeli drzewiastych. Globalną wagność cechy zdefiniowano jako średnią wartość bezwzględną SHAP po wszystkich próbkach i klasach:

$$I_j = \frac{1}{NC} \sum_{i=1}^N \sum_{c=1}^C |\phi_{ijc}|, \quad (1)$$

gdzie ϕ_{ijc} jest wartością SHAP cechy j dla próbki i i klasy c . Wartości obliczono na losowej próbce 2000 rekordów ze zbioru testowego. Zgodność rankingów dwóch modeli zmierzono współczynnikiem Jaccarda na zbiorach 15 najważniejszych cech.

E. Środowisko i wydajność

Eksperymenty przeprowadzono na komputerze z procesorem AMD Ryzen 7 7700 (8 rdzeni/16 wątków)

Macierz pomyłek - RandomForest (UNSW-NB15)

Analysis	11	416	72	54	103	14	4	0	3	0
Backdoor	11	349	78	9	110	14	0	3	9	0
DoS	103	2424	659	554	216	14	6	47	63	3
Exploits	143	2507	384	7090	412	34	39	351	165	7
Fuzzers	25	815	179	298	4125	29	215	14	362	0
Generic	11	18	97	408	73	18223	0	5	31	5
Normal	1533	1	63	569	11227	3	23351	10	240	3
Reconnaissance	13	287	25	128	38	0	3	2927	74	1
Shellcode	0	1	3	16	39	0	0	1	318	0
Worms	0	0	0	13	1	1	0	0	0	29

True label

Predicted label

Rysunek 2. Macierz pomyłek modelu Random Forest na zbiorze UNSW-NB15. Widoczne mylenie klas *Analysis/Backdoor/DoS/Exploits* oraz *Normal/Fuzzers*.

z wykorzystaniem bibliotek `scikit-learn` 1.9, `xgboost` 3.2 oraz `shap` 0.52. Trening modeli jest mało kosztowny obliczeniowo: Random Forest uczy się w 1,6–1,8 s, a XGBoost w 6,7–9,7 s; klasyfikacja całego zbioru testowego (do 268 tys. próbek) trwa poniżej 0,6 s. Dominującym kosztem obliczeniowym okazuje się natomiast generowanie wyjaśnień: dokładny TreeSHAP dla Random Forest (100 drzew, 10 klas) na próbce 2000 rekordów wymaga rzędu dziesiątek minut i, w przeciwieństwie do treningu XGBoost, nie korzysta z akceleracji GPU. Stanowi to praktyczne ograniczenie stosowania XAI w trybie zbliżonym do czasu rzeczywistego i wskazuje, że wyjaśnienia globalne warto liczyć offline.

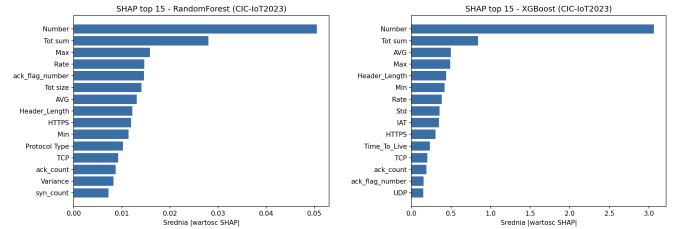
IV. WYNIKI

A. Skuteczność klasyfikacji

Tabela II przedstawia skuteczność obu modeli na obu zbiorach przy użyciu wszystkich cech. XGBoost osiąga nieznacznie wyższą skuteczność niż Random Forest na obu zbiorach (m.in. $F1_m$ 0,534 wobec 0,503 dla UNSW-NB15). Skuteczność na CIC-IoT2023 (dokładność $\approx 83\%$) jest wyraźnie wyższa niż na UNSW-NB15 ($\approx 70\%$), co odzwierciedla trudność oficjalnego podziału UNSW-NB15 (rozbieżność rozkładów zbioru treningowego i testowego). Znaczna różnica między $F1$ makro a $F1$ ważoną na UNSW-NB15 (0,503 wobec 0,743 dla Random Forest) wynika z silnego niezrównoważenia klas: model poprawnie klasyfikuje klasy liczne, lecz słabo radzi sobie z nielicznymi (np. *Worms*, *Shellcode*, *Backdoor*). Macierz pomyłek (Rys. 2) ujawnia źródło błędów: klasy semantycznie zbliżone (*Analysis*, *Backdoor*, *DoS* i *Exploits*) są ze sobą mylone, a część ruchu normalnego klasyfikowana jest jako *Fuzzers*. Jest to znana trudność zbioru UNSW-NB15, wynikająca z nakładania się rozkładów cech tych kategorii.

Tabela II
SKUTECZNOŚĆ KLASYFIKACJI WIELOKLASOWEJ (WSZYSTKIE CECHY).

Zbiór	Model	Acc.	Prec. _m	Rec. _m	F1 _m
UNSW-NB15	Random Forest	0,693	0,522	0,603	0,503
	XGBoost	0,713	0,548	0,630	0,534
CIC-IoT2023	Random Forest	0,829	0,762	0,717	0,732
	XGBoost	0,831	0,736	0,737	0,735



Rysunek 3. Globalna ważność cech wg SHAP dla Random Forest (po lewej) i XGBoost (po prawej) na zbiorze CIC-IoT2023.

Skróty: Acc. to dokładność, a Prec._m/Rec._m/F1_m to precyzja, czułość i F1 uśrednione makro (jednakowa waga klas). Wartości najlepsze w obrębie zbioru pogrubiono.

B. Zgodność wyjaśnień Random Forest i XGBoost

Rysunek 3 przedstawia 15 najważniejszych cech wskazanych przez SHAP dla obu modeli na zbiorze CIC-IoT2023. W obu modelach czołowe miejsca zajmują te same cechy: liczba pakietów (Number), sumaryczny wolumen bajtów (Tot sum) oraz statystyki rozmiaru pakietów (Max, Min, AVG) i tempo transmisji (Rate). Dla UNSW-NB15 dominują analogiczne atrybuty: sbytes, smean, sttl oraz dbytes/dload. Zgodność rankingów (współczynnik Jaccarda dla 15 najważniejszych cech) wynosi 0,667 dla UNSW-NB15 oraz 0,579 dla CIC-IoT2023 (Tabela III); zbiory cech wspólnych liczą odpowiednio 12 i 11 atrybutów.

C. Selekcja cech na podstawie SHAP

Aby zweryfikować obserwację z pracy [10], ponownie wytrenowano modele na 15 cechach o najwyższej ważności SHAP. Tabela IV pokazuje, że redukcja wymiarowości z 42/39 do 15 cech praktycznie nie pogarsza skuteczności: dla UNSW-NB15 wyniki pozostają niemal identyczne (a dla Random Forest nawet nieznacznie rosną, z 0,693 do 0,709 dokładności), natomiast dla CIC-IoT2023 spadek jest niewielki (3–4 punkty procentowe dokładności). Potwierdza to obserwację z [10], że cechy wskazane przez SHAP niosą większość informacji istotnej dla klasyfikacji, a XAI może służyć jako skuteczna metoda selekcji cech.

D. Spójność wyjaśnień pomiędzy zbiorami

Mimo odmiennych zestawów cech obu zbiorów, najważniejsze atrybuty należą w obu przypadkach do tych samych kategorii semantycznych. W obu zbiorach dominują cechy opisujące wolumen przesyłanych danych (sbytes, dbytes w UNSW-NB15; Tot sum, Tot

Tabela III
ZGODNOŚĆ 15 NAJWAŻNIEJSZYCH CECH SHAP: RANDOM FOREST VS XGBOOST.

Zbiór	Jaccard (top-15)	Cechy wspólne
UNSW-NB15	0,667	12 z 15
CIC-IoT2023	0,579	11 z 15

Tabela IV
WPŁYW OGRANICZENIA DO 15 CECH SHAP NA SKUTECZNOŚĆ (DOKŁADNOŚĆ / F1_M).

Zbiór	Model	Wszystkie cechy	15 cech SHAP
UNSW-NB15	Random Forest	0,693 / 0,503	0,709 / 0,525
	XGBoost	0,713 / 0,534	0,705 / 0,533
CIC-IoT2023	Random Forest	0,829 / 0,732	0,795 / 0,683
	XGBoost	0,831 / 0,735	0,790 / 0,694

size w CIC-IoT2023), rozmiar pakietów (smean, dmean; Min, Max, AVG) oraz charakterystykę połączenia (dload, sttl; Rate, Header_Length). Wskazuje to, że pomimo odmiennego pochodzenia ruchu (sieć ogólna vs IoT) modele opierają decyzje na tych samych zjawiskach: nietypowym wolumenie i rozmiarze ruchu, charakterystycznym dla ataków wolumetrycznych (DoS/DDoS, skanowanie).

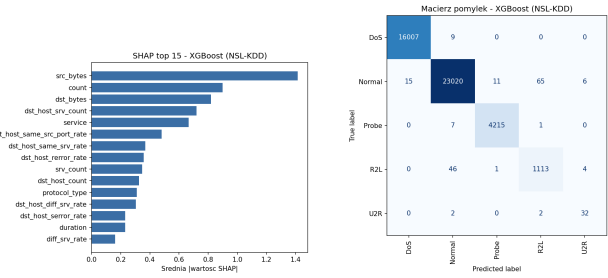
V. DYSKUSJA

A. Walidacja reprodukcyjna na NSL-KDD

Aby zweryfikować, że implementacja jest poprawna i że niska skuteczność na trudnych zbiorach nie wynika z błędu, uruchomiono ten sam potok na zbiorze NSL-KDD, wykorzystanym również przez autorów pracy bazowej. Przy protokole losowego podziału 70/30 oba modele osiągają wyniki na poziomie raportowanym w literaturze: Random Forest uzyskuje dokładność 0,994 i F1 ważoną 0,994, a XGBoost odpowiednio 0,996 i 0,996 (Tabela V). Praca bazowa [1] podaje dla tego zbioru dokładność rzędu 0,99, zatem zastosowany potok wiernie odtwarza opublikowane wyniki na wspólnym zbiorze danych. Niższa wartość F1 makro (0,93–0,95) wynika z klasy U2R, reprezentowanej przez zaledwie kilkadziesiąt próbek, co stanowi znaną trudność zbioru NSL-KDD, niezależną od implementacji. Wynik ten potwierdza, że różnice w skuteczności pomiędzy zbiorami (Tab. II) są właściwością danych, a nie artefaktem implementacji. Macierz pomyłek (Rys. 4, po prawej) wykazuje niemal idealną diagonalę, a nieliczne błędy dotyczą wyłącznie klas R2L i U2R mylonych z ruchem Normal. Co istotne, ranking SHAP (Rys. 4, po lewej) wskazuje te same kategorie cech, co na zbiorach głównych, czyli wolumen bajtów (src_bytes, dst_bytes) oraz liczniki połączeń (count, dst_host_srv_count), co dodatkowo potwierdza spójność wyjaśnień pomiędzy zbiorami danych.

B. Porównanie z pracą bazową

Praca bazowa [1] raportuje dokładność około 99,1% oraz F1 około 98,7% na zbiorach NSL-KDD i CICIDS-2017, a jej rozszerzenie [10] podaje dla Random Forest



Rysunek 4. Walidacja reprodukcyjna na NSL-KDD (XGBoost): globalna ważność cech wg SHAP (po lewej) oraz macierz pomyłek (po prawej). Niemal idealna diagonalna potwierdza dokładność 0,996; jedyne istotne pomyłki dotyczą nielicznych klas R2L i U2R.

Tabela V
POZYCJONOWANIE MODELI WZGLĘDEM PRACY BAZOWEJ. F1 PODANO W WARIANCIE WAŻONYM (PORÓWNYWALNYM Z PRACĄ BAZOWĄ); F1 MAKRO ZNAJDUJE SIĘ W TAB. II.

Źródło / zbiór	Model	Kl.	Acc	F1 _w
<i>Praca bazowa [1], [10]</i>				
RoEduNet-SIMARGL2021	RF	3	0,999	0,999
CICIDS-2017	RF	7	0,988	0,961
NSL-KDD/CICIDS (najl.)	ens.	–	0,991	0,987
<i>Niniejsza praca</i>				
NSL-KDD †	RF	5	0,994	0,994
NSL-KDD †	XGBoost	5	0,996	0,996
UNSW-NB15	RF	10	0,693	0,743
UNSW-NB15	XGBoost	10	0,713	0,760
CIC-IoT2023	RF	8	0,829	0,828
CIC-IoT2023	XGBoost	8	0,831	0,832

† Walidacja reprodukcyjna na wspólnym zbiorze z pracą bazową.

F1 równe 0,999 (RoEduNet-SIMARGL2021) oraz 0,961 (CICIDS-2017). Tabela V pozycjonuje uzyskane modele względem tych wyników. Na wspólnym zbiorze (NSL-KDD) porównanie jest bezpośrednie i wykazuje pełną zgodność: uzyskane modele dorównują pracy bazowej, a nawet ją nieznacznie przewyższają (0,994–0,996 wobec ≈0,99). Dla zbiorów UNSW-NB15 i CIC-IoT2023 porównanie jest jedynie orientacyjne: praca bazowa korzystała z łatwiejszych zbiorów o mniejszej liczbie klas (RoEduNet ma 3 klasy) i pozbawionych rozbieżności rozkładów zbioru treningowego i testowego, charakterystycznej dla UNSW-NB15. Po sprowadzeniu do porównywalnej metryki (F1 ważona) uzyskany wynik na CIC-IoT2023 (0,832) zbliża się do wyniku pracy bazowej na CICIDS-2017 (0,961), mimo użycia nowszego i trudniejszego zbioru. Niższa skuteczność na UNSW-NB15 jest zgodna z literaturą i wynika z bardziej zróżnicowanego rozkładu klas oraz silnego niezrównoważenia (np. klasy Worms czy Shellcode).

W zakresie wyjaśnień uzyskane obserwacje potwierdzają wnioski pracy bazowej oraz [10]: SHAP konsekwentnie wskazuje cechy związane z wolumenem przesyłanych danych (liczba i suma bajtów, rozmiar pakietów), tempem transmisji oraz czasem trwania połączenia. W pracy bazowej dla CICIDS-2017 najważniejsze były m.in. Destination

Port, Init_Win_Bytes_Backward oraz statystyki długości pakietów, semantycznie odpowiadające cechom dominującym w przeprowadzonych eksperymentach.

C. Zgodność modeli a zaufanie

Wysoka zgodność rankingów SHAP pomiędzy Random Forest a XGBoost jest istotna z punktu widzenia bezpieczeństwa: skoro dwa niezależne modele opierają decyzje na tych samych cechach ruchu, wyjaśnienia są bardziej wiarygodne i mniej zależne od wyboru algorytmu. Stanowi to praktyczną przesłankę zaufania do systemu NIDS.

D. Ograniczenia

Analiza obejmuje dwa modele drzewiaste; nie zbadano modeli głębokich ani metody LIME. Próbkowanie zbioru CIC-IoT2023 oraz obliczanie SHAP na podzbiorze testowym mogą nieznacznie wpływać na rankingi. Kodowanie etykietowe cech kategorycznych w UNSW-NB15 upraszcza interpretację, lecz wprowadza sztuczny porządek wartości. Istotną wadą praktycznego wykorzystania jest długi czas generowania wyjaśnień SHAP, który utrudnia stosowanie metody w trybie zbliżonym do czasu rzeczywistego. Jego skrócenie umożliwiłaby implementacja obliczeń na CUDA lub dalsze ograniczanie liczby cech.

VI. WNIOSKI

Zaimplementowano prostą metodykę wyjaśnialnego wykrywania włamań, opartą na modelach Random Forest i XGBoost oraz metodzie SHAP, i przebadano ją na dwóch współczesnych zbiorach danych. Oba modele osiągają wysoką skuteczność klasyfikacji wieloklasowej i, co kluczowe, wskazują w dużej mierze te same cechy jako najważniejsze, co potwierdza wiarygodność wyjaśnień. Dominują cechy związane z wolumenem bajtów, rozmiarem pakietów i czasem trwania połączenia, zgodnie z obserwacjami z literatury, również pomiędzy odmiennymi zbiorami danych. Ograniczenie do 15 cech wskazanych przez SHAP zachowuje skuteczność modeli, co potwierdza użyteczność XAI nie tylko w celach interpretacyjnych, lecz również w redukcji wymiarowości. Dalsze prace mogą stanowić: rozszerzenie analizy o metodę LIME oraz modele głębokie, a także o lokalne wyjaśnienia pojedynczych alarmów.

LITERATURA

- [1] O. Arreche, T. R. Guntur, J. W. Roberts, and M. Abdallah, "XAI-IDS: Toward proposing an explainable artificial intelligence framework for enhancing network intrusion detection systems," *Applied Sciences*, vol. 14, no. 10, p. 4170, 2024.
- [2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [5] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.
- [6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015, pp. 1–6.
- [7] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [8] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," *IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pp. 1–6, 2009.
- [9] M. T. Ribeiro, S. Singh, and C. Guestrin, "'why should i trust you?': Explaining the predictions of any classifier," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.
- [10] O. Arreche, I. Bibers, and M. Abdallah, "XAI-based feature selection for improved network intrusion detection systems," *arXiv preprint arXiv:2410.10050*, 2024.
- [11] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.